# efficient-sumcheck: Canonical Redesign

A trait-based sumcheck library grounded in Thaler's formalization.

## Slide 1: The Problem

The library grew to **10,800 LOC** through incremental SIMD + WHIR integration.

- 3 separate protocol runners (multilinear, inner-product, coefficient)
- 4 order strategies (ascending, descending, graycode, MSB)
- Old `Prover` trait with 6 associated types
- SIMD dispatch graph not wired to the new MSB entry points (~2,000 lines unused)
- Duplicate `_with_hook` entry points

**The code was correct. The architecture was not.**

# Slide 2: The Goal

Reduce complexity without losing functionality.

| Metric | Before | After |
|---|---|---|
| Lines of code | 10,800 | 7,448 |
| Source files | ~100 | 43 |
| Public entry points | 10+ | 4 |
| Order strategies | 4 | 1 (MSB) |
| Protocol runners | 3 | 1 |
| Verifiers | 2 | 1 |
| Return types | 2 | 1 |
| Tests | 69 | 63 |
| Clippy warnings | 46 | 0 |

## Slide 3: The Authoritative Source

Justin Thaler, *Proofs, Arguments, and Zero-Knowledge*, Chapter 4.

**Proposition 4.1.** Given a v-variate polynomial g over F with degree at most d in each variable, the sum-check protocol proves `H = sum_{b in {0,1}^v} g(b)` in v rounds.

- Completeness error: 0
- Soundness error: <= v * d / |F|

**Key insight:** the protocol is *one* protocol parameterized by g. Three "different" sumchecks are three instantiations.

# Slide 4: One Protocol, Three Instantiations

| Use case | g | Degree | Reference |
|---|---|---|---|
| Multilinear | f_tilde (MLE) | 1 | Thaler S4.1 |
| Inner product | f_tilde * g_tilde | 2 | Thaler S4.4 |
| Coefficient | user-defined | d | Thaler S4.6 |

The protocol (transcript, consistency checks, challenges) is identical.

Only the prover's round polynomial computation changes.

This motivates **one runner + one trait**, not three functions.

# Slide 5: The Prover Trait

```rust
pub trait SumcheckProver<F: SumcheckField> {
    fn degree(&self) -> usize;
    fn round(&mut self, challenge: Option<F>) -> Vec<F>;
    fn finalize(&mut self, last_challenge: F);
    fn final_value(&self) -> F;
}
```

## Lifecycle:

```
round(None)          -> g_0 evaluations     // round 0
round(Some(r_0))     -> g_1 evaluations     // fold with r_0, compute g_1
...
round(Some(r_{v-2})) -> g_{v-1} evaluations // fold, compute
finalize(r_{v-1})                           // apply last challenge
final_value()        -> g(r_0, ..., r_{v-1}) // oracle value
```

# Slide 6: The Protocol Runner

```rust
pub fn sumcheck<F, T, H, P>(
    prover: &mut P,
    num_rounds: usize,
    transcript: &mut T,
    hook: H,
) -> SumcheckProof<F>
```

One function handles:

- Full sumcheck (num_rounds = v)
- Partial sumcheck (num_rounds < v) for GKR and WHIR
- Per-round hooks for proof-of-work grinding
- Any polynomial degree (degree is prover-reported)

**The runner never inspects prover internals.**

# Slide 7: The Verifier

```rust
pub fn sumcheck_verify<F, T, H>(
    claimed_sum: F,
    expected_degree: usize,
    num_rounds: usize,
    transcript: &mut T,
    hook: H,
) -> Result<(F, Vec<F>), SumcheckError>
```

- Checks g_j(0) + g_j(1) = claim each round
- Evaluates g_j(r_j) via Lagrange interpolation (any degree)
- Does NOT perform the final oracle check (Thaler Remark 4.2)
- Returns (final_claim, challenges) for caller to verify

## Slide 8: Unified Proof Type

```rust
pub struct SumcheckProof<F: SumcheckField> {
    pub round_polys: Vec<Vec<F>>,  // g_j at {0,1,...,d}
    pub challenges: Vec<F>,        // r_1, ..., r_v
    pub final_value: F,            // g(r_1, ..., r_v)
}
```

Replaces both `Sumcheck<F>` and `ProductSumcheck<F>`.

Prover-specific post-state (e.g., `(f(r), g(r))` for inner product) lives on the prover via `&mut P` ownership, not in the proof.

# Slide 9: Concrete Provers

## MultilinearProver (degree 1)

```
let mut prover = MultilinearProver::new(evals);
let proof = sumcheck(&mut prover, v, &mut t, |_, _| {});
```

## InnerProductProver (degree 2)

```
let mut prover = InnerProductProver::new(a, b);
let proof = sumcheck(&mut prover, v, &mut t, |_, _| {});
let (f_r, g_r) = prover.final_evaluations();
```

Same runner. Same verifier. Same proof type.

## Slide 10: The Fold Primitive (Lemma 4.3)

```
new[k] = v[k] + weight * (v[k + L/2] - v[k])
```

- Half-split (MSB) layout: fold the topmost variable each round
- Matches Thaler eq. 4.13 directly
- Non-power-of-two: implicit zero padding on the high half
- SIMD-accelerated for Goldilocks (transparent, zero overhead otherwise)
- Exposed publicly for WHIR's `multilinear_fold`

# Slide 11: SIMD Acceleration

Goldilocks field (p = 2^64 - 2^32 + 1):

| Backend | Width | Platform |
|---|---|---|
| NEON | 2-wide | aarch64 (Apple M-series, Graviton) |
| AVX-512 IFMA | 8-wide | x86_64 (Sapphire Rapids) |

Detection via `SumcheckField::_simd_field_config()`:

- Returns `Some(SimdFieldConfig { modulus, element_bytes })` for Goldilocks
- Returns `None` for everything else
- LLVM const-folds the branch after monomorphization
- **Zero overhead on non-Goldilocks fields**

# Slide 12: Generic Field Trait

```rust
pub trait SumcheckField:
    Copy + Send + Sync + PartialEq + Debug
    + Add + Sub + Mul + Neg
    + AddAssign + SubAssign + MulAssign
    + Sum + 'static
{
    const ZERO: Self;
    const ONE: Self;
    fn from_u64(val: u64) -> Self;
    fn inverse(&self) -> Option<Self>;
    fn extension_degree() -> u64;
    fn _simd_field_config() -> Option<SimdFieldConfig>;
}
```

- **Not coupled to arkworks.** Any field-like type works.
- Blanket impl for `ark_ff::Field` behind `feature = "arkworks"` (default-on).
- Non-arkworks Goldilocks can opt into SIMD by overriding `_simd_field_config()`.

## Slide 13: Cross-Field Support (BF -> EF)

Evaluations in base field BF (e.g., Goldilocks).
Challenges from extension field EF (e.g., Goldilocks^3) for soundness.

```
pub trait ExtensionOf<BF: SumcheckField>: SumcheckField + From<BF> {}
```

The transition is prover-internal:

- Round 0: compute over BF
- `round(Some(r_1))`: lift BF -> EF, continue in EF
- Protocol runner and verifier never see BF

# Slide 14: WHIR Integration

```
for round_config in &self.round_configs {
    round_config.committer.commit(&a);
    update_covector(&mut b, &stir_challenges);

    let proof = sumcheck(
        &mut InnerProductProver::new(a, b),
        round_config.folding_factor,     // partial!
        &mut transcript,
        |_, t| round_config.round_pow.prove(t),  // hook!
    );

    a = prover.a();  // post-state access
    b = prover.b();
}
```

Three features exercised: partial execution, per-round hook, post-state.

# Slide 15: GKR Integration

GKR runs d sumcheck invocations (one per circuit layer).

```
let proof = sumcheck(
    &mut gkr_prover,
    num_rounds,
    &mut t,
    |_, _| {},
);
let (w_b, w_c) = gkr_prover.claimed_w_values();
```

Then reduce-to-one (separate sub-protocol, not baked into sumcheck):

```
let (point, value) = reduce_to_one(b, c, v0, v1, &mut t);
```

Composable building blocks, not monolithic protocols.

## Slide 16: Three Prover Strategies

| Strategy | Space | Time | Input | Status |
|---|---|---|---|---|
| Time | O(2^v) | O(2^v) | Vec | Implemented |
| Space | O(v) | O(v * 2^v) | Stream | Framework ready |
| Blendy | O(2^k) | O(2^v) | Stream | Deferred (LSB investigation) |

**Blendy note:** Jolt found LSB ordering optimal for Lagrange polynomial updates. Investigating whether internal LSB traversal can be decoupled from the MSB round ordering before implementing.

All three produce identical round polynomials.

# Slide 17: Advanced Optimizations Fit the Trait

From Bagad-Dao-Domb-Thaler (ePrint 2025/1117):

| Optimization | Changes protocol? | Changes wire format? |
|---|---|---|
| LinearTime (Alg 1) | No | No |
| SqrtSpace (Alg 2) | No | No |
| SmallValue (Alg 3-4) | No | No |
| EqPoly (Alg 5-6) | No | No |
| Univariate skip | No | No |

**All optimizations live below the trait boundary.**

The protocol runner, verifier, and transcript are untouched.
Each optimization is a different `SumcheckProver` implementation.

# Slide 18: What We Deleted

| Module | LOC | Why |
|---|---|---|
| multilinear/provers/ | 1,190 | Old Prover trait scaffolding |
| multilinear_product/provers/ | 1,175 | Same |
| order_strategy/ | 315 | 4 strategies -> MSB only |
| messages/ | 130 | Unused |
| interpolation/ | 197 | Graycode-coupled (blendy dependency) |
| tests/multilinear/ | ~400 | Old test harness |
| tests/multilinear_product/ | ~400 | Same |
| prover/core.rs | 24 | Old trait definition |
| Various dead SIMD code | ~500 | Orphaned dispatch paths |

**Total: 4,518 lines deleted across 86 files.**

# Slide 19: Feature Gate Architecture

```
[features]
default = ["arkworks", "parallel"]

arkworks = ["ark-ff", "ark-poly", "ark-serialize",
            "ark-std", "spongefish"]
parallel = ["rayon", "ark-ff?/parallel", ...]
```

- `--no-default-features`: pure `SumcheckField` library, no arkworks
- `--features arkworks`: blanket impl for `ark_ff::Field`
- `--features parallel`: rayon parallelism for fold and round computation
- SIMD: always compiled in, dispatched at const-fold time

# Slide 20: Benchmarking Strategy

## Layer 1: Kernel Throughput

- fold elements/sec vs memory bandwidth ceiling
- round() overhead on top of fold
- BF->EF promotion cliff

## Layer 2: Protocol Scaling

- Full sumcheck: time vs 2^v on log-log axes
- Matrix: {time, blendy} x {ml, IP} x {F64, F64Ext3} x {2^16..2^24}
- Key metric: **time per element per round**

## Layer 3: Downstream Integration

- WHIR sumcheck bench as acceptance gate
- Regression detection, not absolute performance

# Slide 21: CI Benchmark Infrastructure

```yaml
# .github/workflows/bench.yml
- name: Detect CPU features
  run: |
    if grep -q avx512ifma /proc/cpuinfo; then
      echo "rustflags=-C target-feature=+avx512ifma"
    fi

- name: Run benchmarks
  run: cargo bench --bench sumcheck -- --output-format bencher

- uses: benchmark-action/github-action-benchmark@v1
  with:
    alert-threshold: "115%"
    auto-push: true  # on main
```

- Auto-detects AVX-512 IFMA / NEON / scalar
- Results to gh-pages with trendlines
- 15% regression = alert + PR comment

## Slide 22: Current Benchmark Matrix

18 benchmark points across 6 groups:

```
multilinear/F64:      2^16, 2^20, 2^24
multilinear/F64Ext3:  2^16, 2^20, 2^24
inner_product/F64:    2^16, 2^20, 2^24
inner_product/F64Ext3: 2^16, 2^20, 2^24
fold/F64:             2^16, 2^20, 2^24
fold/F64Ext3:         2^16, 2^20, 2^24
```

Criterion harness with `Throughput::Elements(n)` annotations.

~5 minutes on AVX-512 hardware.

## Slide 23: Migration Table

| Old | New |
|---|---|
| `multilinear_sumcheck(evals, t, hook)` | `sumcheck(&mut MultilinearProver::new(evals), v, t, hook)` |
| `inner_product_sumcheck(a, b, t, hook)` | `sumcheck(&mut InnerProductProver::new(a, b), v, t, hook)` |
| `multilinear_sumcheck_partial(...)` | `sumcheck(&mut prover, k, t, hook)` |
| `fold(values, weight)` | `fold(values, weight)` (unchanged) |
| `Sumcheck<F>` | `SumcheckProof<F>` |
| `ProductSumcheck<F>` | `SumcheckProof<F>` (unified) |

## Slide 24: What's NOT in Scope

Explicit non-goals (from Thaler's framing):

- **Generic IP trait** -- sumcheck is specific, not an instance of a framework
- **Zero-knowledge** -- future `ZkSumcheckProver` wrapper (masking polynomials)
- **Batching** -- compose via random linear combinations externally
- **Reduce-to-one** -- separate sub-protocol (S4.5.2), not part of sumcheck

These can be added later without changing the core trait or runner.

## Slide 25: Design Principles

1. **One protocol, one trait, many implementations.**
   Three polynomial shapes are three `SumcheckProver` impls, not three runners.

2. **The trait boundary is the optimization boundary.**
   Everything above it (runner, verifier, transcript) is fixed.
   Everything below it (fold, SIMD, table layout, multiplication tricks) is freedom.

3. **Partial execution is first-class.**
   `num_rounds < v` enables GKR and WHIR without special-casing.

4. **Post-state via ownership, not return types.**
   `&mut P` survives sumcheck; prover-specific accessors are type-safe.

5. **No final oracle check in the verifier.**
   The caller decides how to verify g(r) -- direct eval, delegation, or commit.

# Slide 26: Next Steps

| Item | Status | Notes |
|---|---|---|
| CoefficientProver | Deferred | Port from coefficient_sumcheck.rs |
| Blendy prover | Deferred | Pending LSB vs MSB investigation (Jolt) |
| Port utilities to SumcheckField | Pending | eq_evals, poly_ops, streams |
| Self-hosted CI runner | Pending | EC2 Sapphire Rapids for AVX-512 |
| WHIR integration test | Pending | Point WHIR at rewrite branch |
| README update | Pending | New API docs + examples |

## Slide 27: Summary

**The sum-check protocol is one protocol.**

We made the code match that fact.

- 1 runner, 1 verifier, 1 proof type, 1 fold
- Generic over any field (not arkworks-specific)
- SIMD transparent for Goldilocks
- 31% fewer lines, 57% fewer files
- All known optimizations fit below the trait boundary
- GKR and WHIR compose naturally
- CI regression tracking from day one